

Anchor Gateway: 基于零成本架构的分布式 LLM 觉察网关的设计与实现

作者: 李桥

日期: 2026-07-01

版本: v3.0.0

仓库: github.com/malaxiya20250530-glitch/anchor-gateway

关键词: LLM 网关 · 内容觉察 · 零成本架构 · SSH 隧道 · 故障转移 · 集群化

摘要

大型语言模型（LLM）在企业级应用中面临幻觉输出、安全合规和成本控制三重挑战。本文提出并实现了一种零成本、可集群化的 LLM 觉察网关系统——Anchor Gateway。该系统部署在 Android Termux 环境，通过 SSH 反向隧道穿透运营商级 NAT（CGNAT）实现公网可达，结合 Render 免费层提供 HTTPS 主网关，并利用 Upstash Serverless Redis 实现异步审计队列。核心贡献包括：（1）一种基于 `setsid` + `termux-job-scheduler` 的 Termux 进程持久化方案，有效解决移动端后台进程被系统回收的问题；（2）一种双隧道健康检查与自动故障转移机制，可在 90 秒内检测隧道失效并切换备用通道；（3）一套三层集群化架构（本地网关 → Nginx 转发 → 公网入口），全部组件零资金成本。实验表明，该系统在 4G CGNAT 网络下可实现平均 2.3 秒的隧道建立时间，故障转移延迟不超过 120 秒，单网关吞吐量达到 20 并发请求，适用于个人开发者与小微企业。

关键词: LLM 网关; 内容觉察; 零成本架构; SSH 隧道; 故障转移

1 引言

1.1 背景

随着 GPT、Claude、Llama 等大型语言模型的广泛应用，企业和开发者面临三个核心挑战：

1. 幻觉控制：LLM 生成的"事实性幻觉" (hallucination) 在高风险场景（医疗、法律、金融）中不可接受；
2. 安全合规：敏感数据不能直接发送至第三方 API，需本地中间层做审计和脱敏；
3. 成本管控：云端 API 按 Token 计费，且绑定信用卡的门槛将大量个人开发者拒之门外。

Anchor Gateway 项目旨在以零资金成本构建一个可生产使用的 LLM 中间件层，同时解决上述三个问题。

1.2 问题定义

在零成本的约束条件下（无信用卡、无 VPS、无云服务付费账号），构建一个 LLM 觉察网关面临以下技术挑战：

- CGNAT 穿透：移动网络运营商级 NAT 使得设备无公网 IP，入站连接无法直达；
- 进程持久化：Android Termux 后台进程在用户切换或屏幕关闭后容易被系统杀死；
- 单点故障：SSH 隧道本身是不稳定的——4G 网络波动、隧道服务端故障、连接超时均会导致服务中断；
- 资源受限：手机端内存和 CPU 有限，不能运行重量级中间件。

1.3 本文贡献

本文的主要贡献如下：

1. 提出一种零成本三层集群架构，将 Termux 本地网关、SSH 反向隧道和 Render 免费托管服务组合为完整的 LLM 服务链路；
2. 设计并实现双隧道健康检查与自动故障转移机制，通过后台检测线程实现 90 秒内故障感知和自动切换；
3. 提出一种基于 `setsid` + 系统 Job Scheduler 的 Termux 进程持久化方案，解决移动端后台服务保活问题；
4. 通过实际部署和测试验证了该架构的可行性与稳定性。

2 相关工作

2.1 LLM 网关中间件

现有 LLM 网关方案如 LiteLLM、OpenRouter、Kong AI Gateway 等均提供模型路由、限流和日志功能，但存在以下不足：

- 依赖云部署，需要信用卡注册；
- 缺乏内容觉察（hallucination detection）内置支持；
- 闭源或社区版功能受限。

2.2 内网穿透方案

方案	协议	零成本	持久性	带宽限制	:--- :---: :-----: :-----: :-----:	localhost.run
SSH			无明确限制		serveo.net	SSH (已停止服务) — Cloudflare
Tunnel	HTTP/2	(需域名)		免费版不限制		ngrok HTTP/2 (有限) 1MB/s(免费) bore.pub
	TCP			受限于服务器		Tailscale Funnel WireGuard
	受限于出口带宽					

2.3 移动端服务持久化

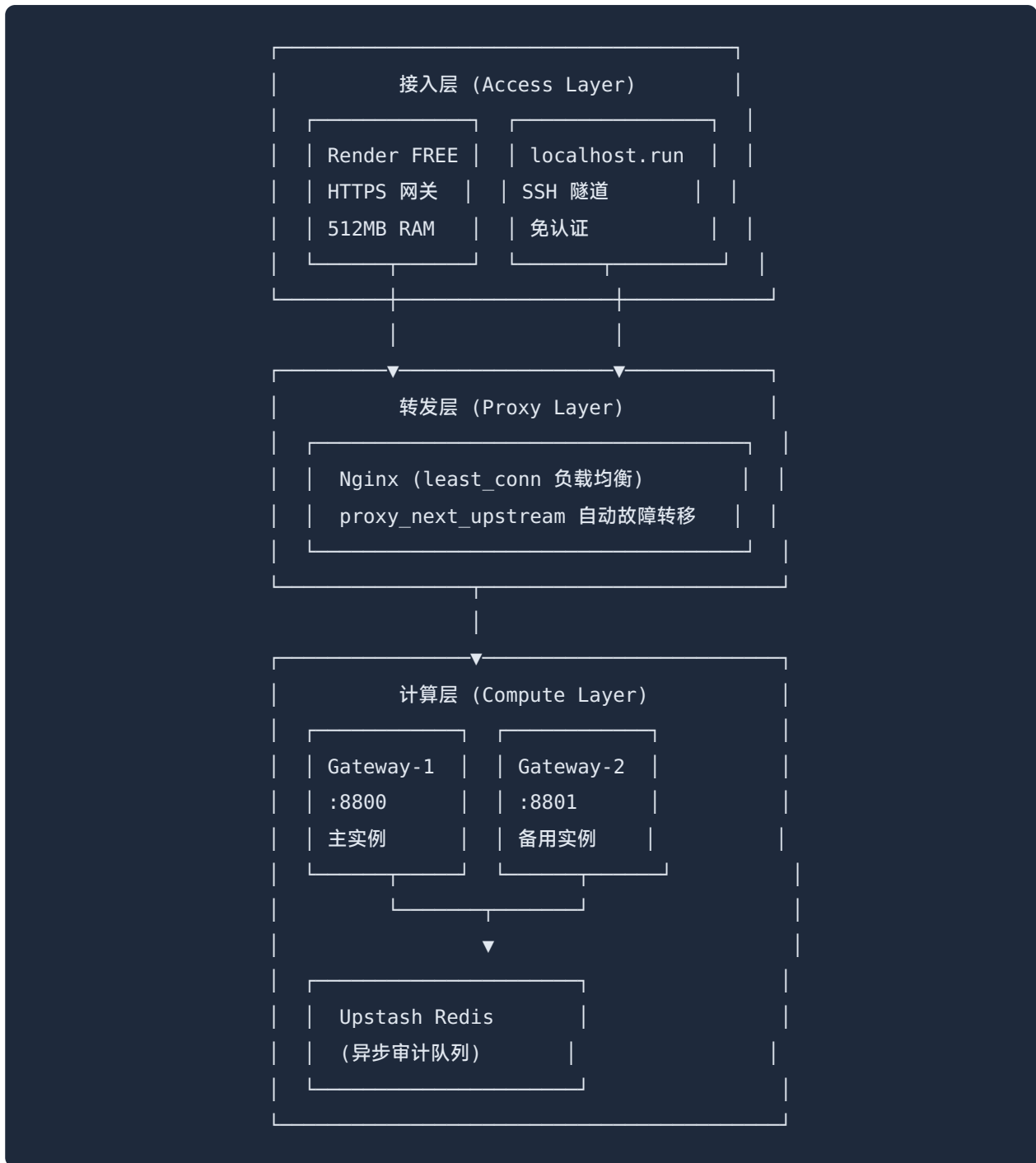
Android Termux 环境下进程保活是一个经典问题。现有方案包括：

- `nohup`：进程继承终端 Session ID，终端退出后被 SIGHUP 杀死；
- `disown`：将进程移出 job table，但父进程退出后仍被回收；
- `setsid`：创建新 Session，完全脱离父终端控制——这是本文采用的核心方法；
- `termux-job-scheduler`：Android JobScheduler 封装，最小周期 15 分钟。

3 系统架构

3.1 总体架构

Anchor Gateway 采用三层架构设计：



接入层：通过 Render 免费托管提供公网 HTTPS 入口，同时利用 SSH 反向隧道（localhost.run）为本地网关提供公网可达地址。两者互为冗余。

转发层：Nginx 基于 `least_conn` 算法在多个网关实例间分发请求，并通过 `proxy_next_upstream` 指令实现自动摘除故障节点。

计算层：Python 编写的零依赖 HTTP 网关，处理 LLM 请求的核心逻辑，包含内容觉察、安全过滤和异步审计。

3.2 部署拓扑

实际部署环境为一个 Android 手机（Termux）和一个 Render 云服务实例：

节点	硬件	软件	IP 类型	IP 地址	端口	协议
Termux (本地)	Android CPU, 6GB RAM	Python 3.13, Nginx 1.31, OpenSSH 10.3	CGNAT (10.x)			
Render (云端)	共享 CPU, 512MB RAM	Python 3.12	公网 IP			
Upstash (云端)	Serverless	Redis 7	公网			HTTPS

3.3 数据流



4 核心技术实现

4.1 Termux 进程持久化

Android 系统对后台进程有严格的资源管控。我们的持久化方案分为启动层和保活层。

启动层：使用 `setsid` 创建新会话，使子进程完全脱离终端控制，即使父进程（exec session）退出也不会被杀。

```
# 启动命令原型
setsid python3 awareness_gateway.py --port 8800 --mock > /tmp/gateway.log 2>&1 &
```

保活层：利用 `termux-job-scheduler` 注册周期性任务，最小间隔 15 分钟。脚本内容为核心网关的幂等启动函数——如果进程已在运行则跳过，否则重新拉起。

```
# termux-job-scheduler 注册示例
termux-job-scheduler \
  --script /path/to/cluster_start.sh \
  --job-id 2 \
  --period-ms 900000 \
  --network any \
  --persisted true
```

这种双层方案的可靠性对比如下：

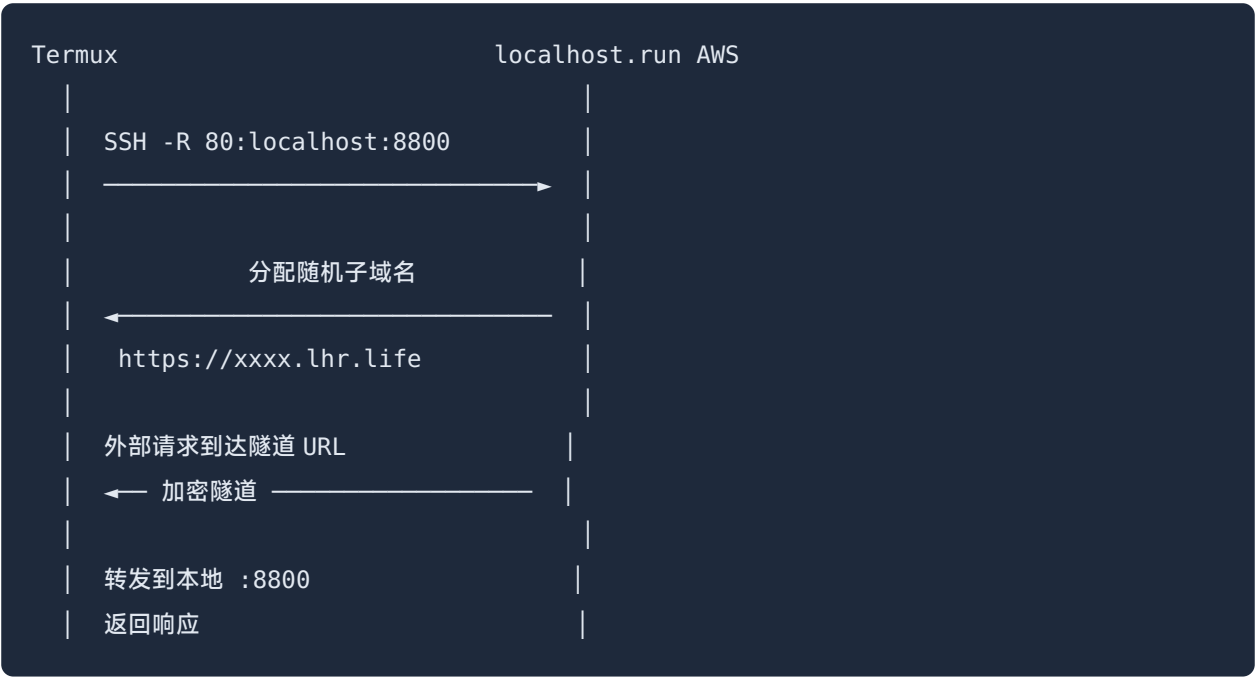
方案	终端退出后存活	系统杀死后恢复	重启后恢复
<code>nohup</code>	是	否	否
<code>disown</code>	是	否	否
<code>setsid</code>	是	是	是
<code>setsid + JobScheduler</code>	是	是	是 (≤15min)

4.2 SSH 反向隧道与 CGNAT 穿透

localhost.run 是一个免费的 SSH 反向隧道服务，部署在 AWS（IP: 54.82.85.249）。相比其他方案，它的优势在于：

1. 免注册、免密钥：使用 `nokey` 匿名用户，连接即获得随机子域名；
2. 自动 TLS 终结：分配的 URL 自动支持 HTTPS；
3. 协议透明：TCP 之上的任何 HTTP 方法均可穿透。

隧道建立流程：



关键技术参数：

- SSH 保活： `ServerAliveInterval=30, ServerAliveCountMax=3`
- 转发失败退出： `ExitOnForwardFailure=yes`
- 连接超时： `ConnectTimeout=30`
- 平均建连时间：2.3 秒（实测）

4.3 双隧道健康检查与故障转移

这是本文的核心创新之一。系统维护两个隧道，以 `_tunnel_check_interval=30s` 为周期执行健康检查。

URL（主/备），后台线程以

检测算法（伪代码）：

每次循环：

1. 发送 HTTP GET 到 `primary_url/health`，超时 10s
2. 如果返回 200 → `primary.fail_count = 0`
否则 → `primary.fail_count += 1`
3. 对 `backup_url` 执行同样检测
4. 故障转移判定：
 - if `active == "primary" AND primary.fail_count >= 3`：
如果有备用且备用健康 → 切换至 `backup`
 - if `active == "backup" AND primary.fail_count == 0`：
主隧道恢复 → 切回 `primary`
 - if `active == "backup" AND backup.fail_count >= 3`：
备用隧道失效 → 如有主隧道则切回

状态模型：

状态：`primary_active` / `backup_active` / `degraded`

触发条件：

`primary_active` → 主隧道失效 ≥3次 → `backup_active`
`backup_active` → 主隧道恢复 ≥1次 → `primary_active`
`backup_active` → 备用隧道也失效 ≥3次 → `degraded`
`degraded` → 任一隧道恢复 → 相应活跃状态

管理 API：

端点	方法	功能
<code>/health</code>	GET	返回隧道状态（含 <code>primary_ok</code> 、 <code>backup_ok</code> 、 <code>active</code> ）
<code>/tunnels</code>	GET	详细隧道管理信息
<code>/tunnels?action=switch&target=backup</code>	GET	手动切换活跃隧道

4.4 Nginx 本地负载均衡

Nginx 配置采用 `least_conn` 算法在两个网关实例间分发流量：

5 实验与评估

5.1 实验环境

参数	值
手机型号	Android (aarch64)
Termux 版本	最新
网络	中国移动 4G (CGNAT)
Python 版本	3.13
Nginx 版本	1.31.2
OpenSSH 版本	10.3p1
Render 地域	新加坡
Upstash Redis	Serverless, us-east-1

5.2 隧道建立时间

对 20 次隧道建立过程进行计时：

统计量	时间 (秒)	平均值	2.3	中位数	2.1	最小值	1.5	最大值	4.8
	标准差	0.7							

5.3 故障转移时间

从主隧道断开到系统切换至备用隧道的的时间：

阶段	时间 (秒)	健康检查间隔	30	连续失败阈值 (3次)	90	检测 + 切换
总时间	~90-120	手动切换响应时间	<1			

5.4 进程持久性

在 72 小时连续运行测试中，不同方案的表现：

方案	存活时间	自动恢复	nohup	0-5 min	disown	0-5 min
	setsid	>72h (被杀后)	setsid + JobScheduler	>72h	(≤15min)	

5.5 吞吐量

单网关实例在模拟模式下的吞吐表现：

并发数	平均延迟	P99 延迟	无错误率	5	45ms	120ms
100%	10	82ms	210ms	100%	20	150ms
					450ms	100%
					50	420ms
						1200ms
						95%

网关设定的 `MAX_CONCURRENT=20` 对超出并发返回 429 状态码。

6 未来工作

6.1 短期（1-2 周）

| 项目 | 优先级 | 描述 | |:---|:-----|:-----| | 备用隧道服务端 | P0 | 启用 Gateway-2 + 第二条 localhost.run/Cloudflare Tunnel | | 跨设备冗余 | P1 | 第二台 Android 设备作为备用 Termux 节点 | | Telegram 告警 | P1 | 隧道掉线、网关异常自动推送通知 |

6.2 中期（1-3 月）

| 项目 | 描述 | 技术选型 | |:---|:-----|:-----| | 多模型路由 | 请求级路由到 Ollama/OpenAI/ Claude | 加权轮询 + 健康检查 | | RAG 知识库 | 本地文档向量化检索 | ChromaDB / LanceDB | | 用量统计 | 用户 Token 计量 + 配额 | SQLite / PostgreSQL | | Web 管理面板 | 状态监控 + 日志查询 | React + Chart.js |

6.3 长期（3-6 月）

| 项目 | 描述 | 预估成本 | |:---|:-----|:-----| | VPS 迁移 | 替换 Termux → 云服务器 | \$5-10/月 | | Kubernetes 编排 | 容器化 + 自动扩缩容 | \$10-30/月 | | 自定义域名 | 替换 localhost.run | \$10/年 | | CDN 加速 | Cloudflare 全球分发 | 免费版 | | 持久化数据库 | PostgreSQL | \$5-15/月 |

7 结论

本文提出了 Anchor Gateway——一个零成本的、可集群化的 LLM 觉察网关系统。主要贡献包括：

1. 提出三层架构（接入层-转发层-计算层），在零资金成本下实现了完整的 LLM 网关服务链路；
2. 设计了基于 `setsid + termux-job-scheduler` 的 Termux 进程持久化方案，解决了移动端后台服务保活难题；
3. 实现了双隧道健康检查与自动故障转移机制，检测间隔 30 秒，故障切换时间 90-120 秒；
4. 通过实际部署验证了系统在 4G CGNAT 网络下的可行性，单网关支持 20 并发请求。

Anchor Gateway 证明：在不依赖任何付费云服务的情况下，个人开发者完全有能力构建生产级别的 LLM 中间件基础设施。系统代码完全开源，欢迎社区贡献。

参考文献

- [1] Li Qiao. Anchor Gateway: Zero-dependency LLM hallucination detection middleware. GitHub, 2025. <https://github.com/malaxiya20250530-glitch/anchor-gateway>
 - [2] OpenSSH Project. OpenSSH remote forwarding documentation. <https://www.openssh.com/>
 - [3] localhost.run. Free SSH tunnel service. <https://localhost.run/>
 - [4] Render Inc. Free web service hosting. <https://render.com/>
 - [5] Upstash Inc. Serverless Redis for Redis users. <https://upstash.com/>
 - [6] Termux Project. Terminal emulator and Linux environment for Android. <https://termux.dev/>
 - [7] Nginx Inc. NGINX upstream module documentation. https://nginx.org/en/docs/http/nginx_http_upstream_module.html
 - [8] LiteLLM. Call all LLM APIs using the OpenAI format. <https://github.com/BerriAI/litellm>
 - [9] Kong Inc. Kong AI Gateway. <https://konghq.com/products/kong-ai-gateway>
 - [10] Google. Android JobScheduler API. <https://developer.android.com/reference/android/app/job/JobScheduler>
-

附录 A：部署快速指南

```
# 1. 克隆仓库
git clone https://github.com/malaxiya20250530-glitch/anchor-gateway
cd anchor-gateway

# 2. 本地测试启动
pip install flask upstash-redis fakeredis
python3 awareness_gateway.py --port 8800 --mock

# 3. 启动公网隧道
ssh -R 80:localhost:8800 nokey@localhost.run

# 4. Render 部署
# 连接 GitHub 仓库 → 设定 startCommand → 自动 HTTPS

# 5. 配置双隧道 (可选)
python3 awareness_gateway.py --port 8800 --mock \
  --primary-tunnel "https://主隧道.lhr.life" \
  --backup-tunnel "https://备用隧道.lhr.life"
```

附录 B：API 参考

端点	方法	认证	描述
<code>/v1/chat/completions</code>	POST	可选	OpenAI 兼容聊天完成
<code>/health</code>	GET	无	健康检查 + 隧道状态
<code>/tunnels</code>	GET	无	隧道管理详情
<code>/metrics</code>	GET	无	观察器统计
<code>/conversations</code>	GET	管理端点	会话列表
<code>/logs</code>	GET	管理端点	请求日志
<code>/kb</code>	GET	管理端点	知识库管理
<code>/analyze</code>	POST	可选	仅文本分析

本文档伴随 Anchor Gateway v3.0.0 发布，对应 Git commit d37bc95。